UNSTRUCTURED DATABASE LAB (CSE12203)

Presented By

- Shreya Ghoshal
  (UG/02/BTCSECSF/2023/003)
- Ayantika Gangopadhyay
  (UG/02/BTCSECSF/2023/004)
- Ujjwal kundu
  (UG/02/BTCSECSF/2023/005)
- Arnab Maji
  (UG/02/BTCSECSF/2023/006)

Guided by : Ms. Anusuya Bera

B.TECH CSE; 3RD YEAR

HOSPITAL MANAGEMENT SYSTEM

PRESENTATION

Design and Implementation using MongoDB for Patient Data Management and Analytics

Date of Submission : 09.04.2026

(NoSQL BASED)

WHAT IS

HOSPITAL MANAGEMENT SYSTEM?

A hospital management system helps store and manage patient and doctor data efficiently. Here we are using MongoDB, which is a NoSQL database, to handle large and flexible data in a better way.

WHY NEEDED?

- To manage large amounts of patient data efficiently
- To reduce manual paperwork and human errors
- To provide quick access to patient history and records
- To schedule and manage appointments
- To improve hospital workflow and coordination
- To ensure data is stored securely and organized properly

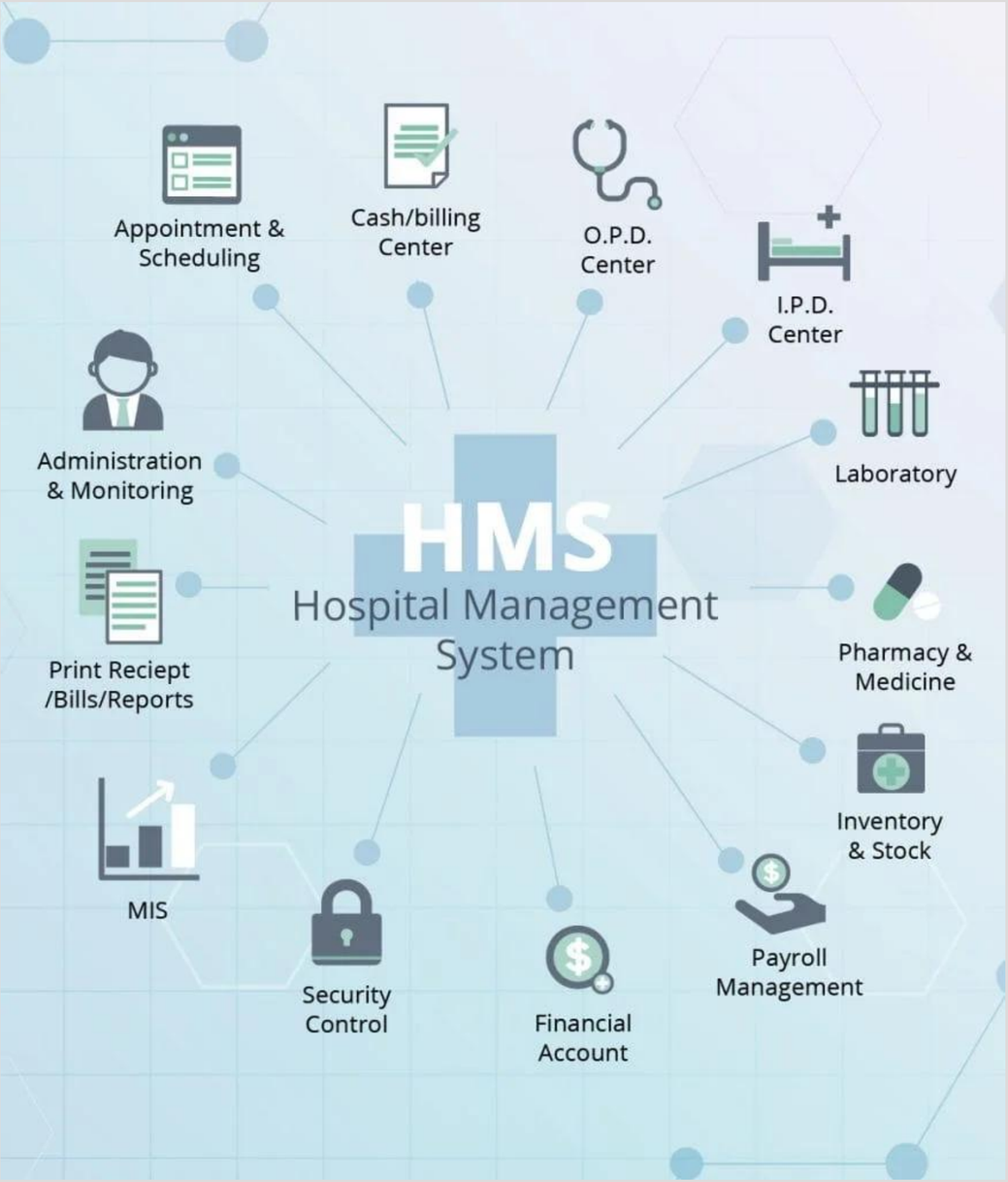
NOSQL:

- NoSQL stands for Not Only SQL – a type of database that stores data in a flexible, non-tabular format.
- Unilife traditional relational databases, it does not require a fixed schema.
  It is designed to handle large volumes of unstructured and semi-structured data.
- In this project, MongoDB is used as a NoSQL database to store hospital data efficiently.

WHAT ARE THE

OBJECTIVES ?

The main objective of this project is to design an efficient system for managing hospital data using a NoSQL database. It aims to store and organize patient records, doctor details, and appointment information in a structured yet flexible manner. The system focuses on implementing CRUD operations for effective data management and using aggregation techniques to generate meaningful insights such as doctor-wise patient analysis. It also demonstrates key NoSQL concepts like embedded documents and indexing to improve performance, scalability, and faster data retrieval. Additionally, the project aims to simulate a real-world hospital environment with better data accessibility and organization.

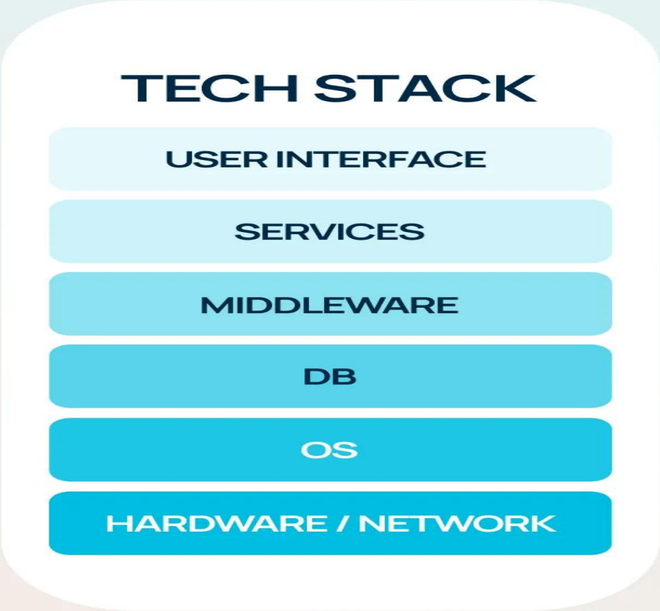


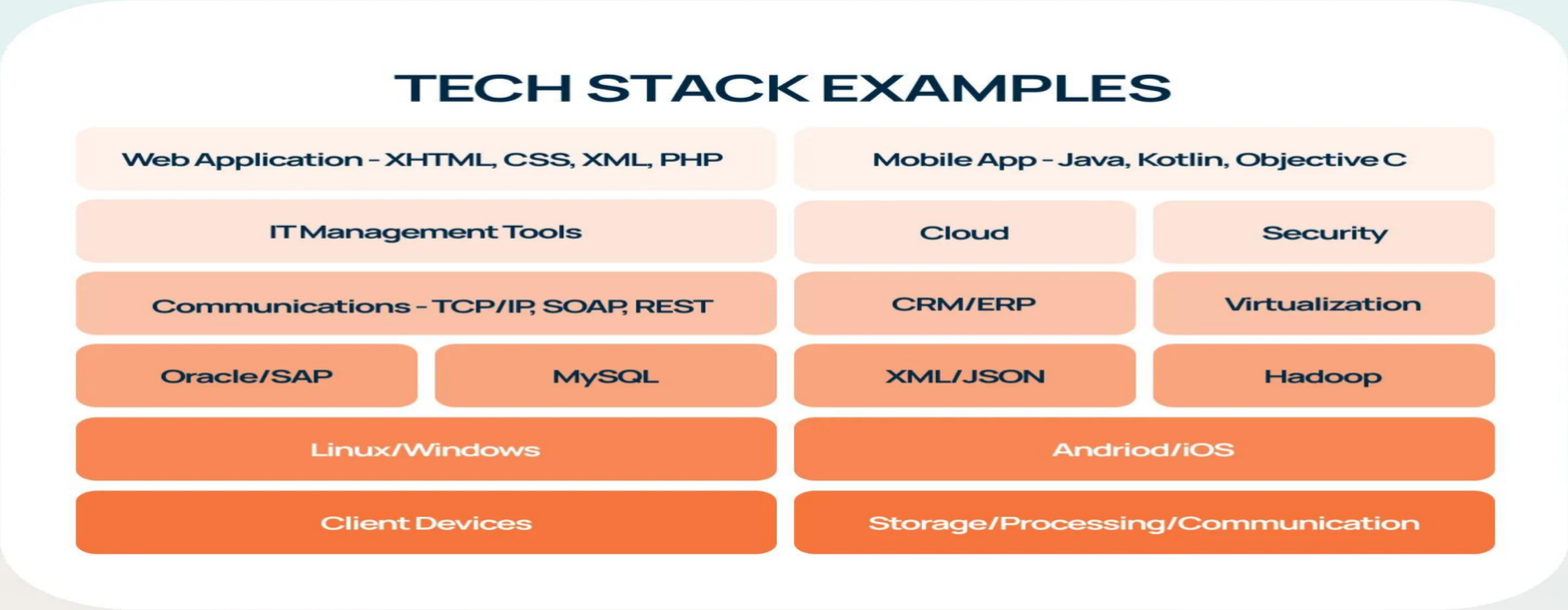
"This project focuses not just on storing data, but on efficient data modeling, performance optimization, and analytical processing using NoSQL concepts."

TECH STACK

- **MongoDb**
✓ Used as the main database for storing flexible and semi-structured hospital data.
- **Studio 3T**
✓ GUI tool for managing MongoDB, executing queries, and visualizing data.
- **MongoDB Query Language (MQL)**
✓ Used for performing CRUD operations, aggregation, and indexing.

- **HTML, CSS, JavaScript**
✓ Used to create a basic user interface for interaction
- **System Environment**
✓ Runs on local machine using MongoDB server





DATA MODELING

"This structured yet flexible data model supports efficient handling of real-world hospital data."

- The system is designed using a **NoSQL data modeling approach** for flexibility and scalability.

- It consists of three main collections:
  Patients → Stores patient details and medical history
  Doctors → Stores doctor information and specialization
  Appointments → Stores appointment records linking patients and doctors

- Relationships are maintained using:
  patient_id (links to patients collection)
  doctor_id (links to doctors collection)

The design ensures **efficient data organization, quick retrieval, and easy scalability.**

.

SCHEMA DESIGN (WITH EMBEDDED DOCUMENTS)

Embedded Documents used for patient history.
Example: Disease, treatment, date inside patient document are stored as an array inside each patient document
Relationships: patient_id → Patients, doctor_id → Doctors

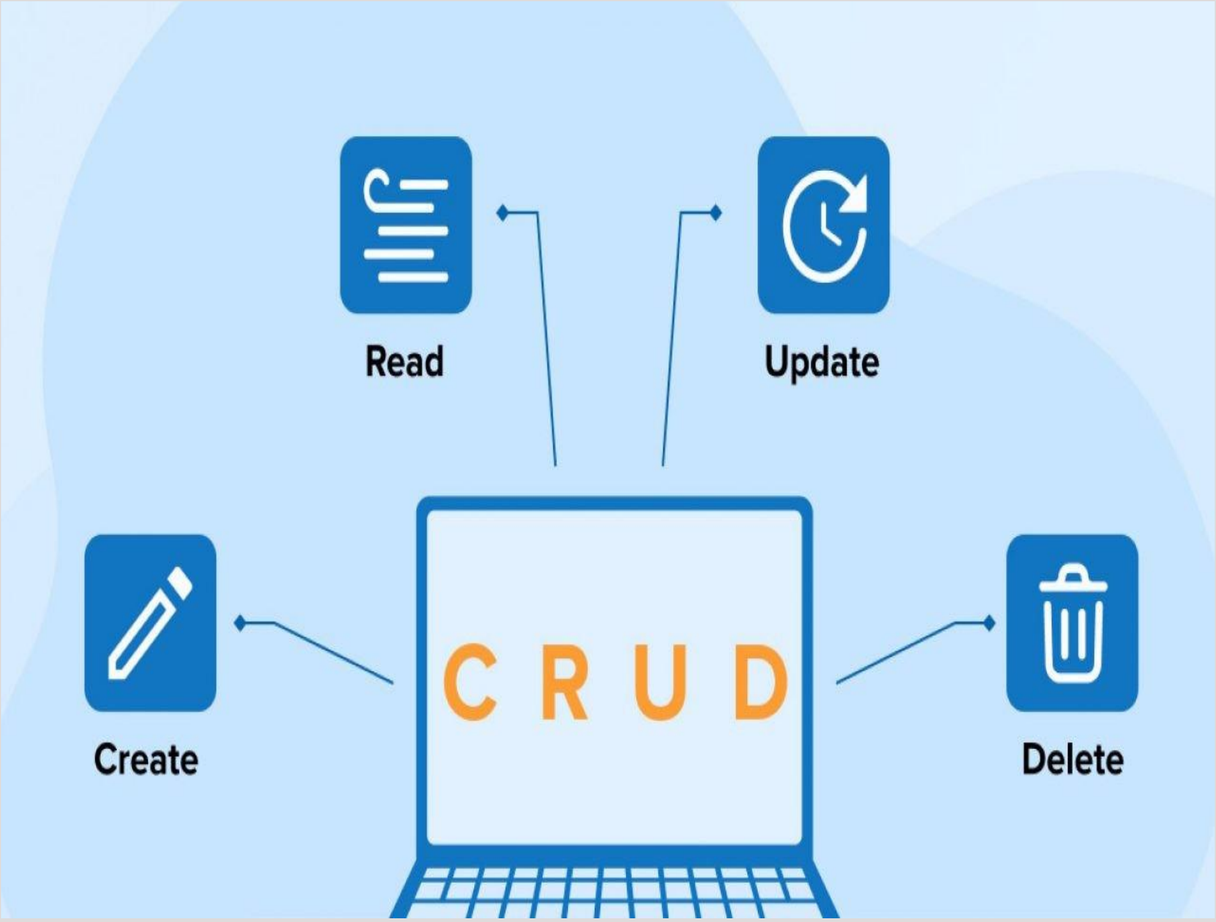
- **Embedded documents** are used to store patient medical history within the patient collection
- The system uses a **flexible schema design** to handle semi-structured healthcare data efficiently
- ➢ Improves read performance by reducing the need for joins.
- This hybrid approach (embedding + referencing) ensures **optimized performance, data consistency, and scalability**

CRUD OPERATIONS

In this system, **CRUD operations are implemented to manage hospital data across patients, doctors, and appointments:**

- **Create (Insert):** New doctors and patients are added to their respective collections, and appointments are created by linking `patient_id` and `doctor_id`. Patient records also include embedded medical history at the time of insertion.
- **Read (Retrieve):** Patient details, doctor information, and appointment records are retrieved using queries. For example, viewing all patients or checking appointments for a specific doctor.
- **Update (Modify):** Existing data is updated as required, such as modifying patient age or adding new medical history using `$ush` within the embedded history array.
- **Delete (Remove):** Appointment records are deleted when no longer needed, ensuring that the database remains clean and up-to-date.

These operations ensure smooth handling of hospital data and maintain accuracy across all collections.



AGGREGATION QUERIES

Aggregation in this system is used to perform advanced data analysis on hospital records. It processes large volumes of appointment data to generate meaningful insights without manual calculation. Using MongoDB's aggregation pipeline, the system groups and analyzes data based on specific fields such as doctor_id.

It is applied to calculate the total number of patients handled by each doctor, helping in understanding workload distribution. Additionally, sorting and limiting operations are used to identify the most visited doctor, providing valuable insights into doctor performance and demand.

These aggregation operations transform raw data into structured analytical results, supporting better decision-making and demonstrating the power of NoSQL in handling real-world data analysis efficiently

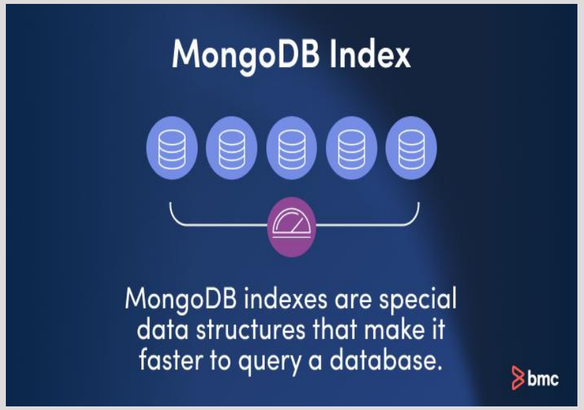
- Patients per Doctor: Counted using $group
- Most Visited Doctor: Identified using $sort and $limit

"Aggregation helps convert raw hospital data into meaningful insights like doctor workload and popularity."

INDEXING

Indexing is implemented to optimize query performance and ensure efficient data retrieval in the system. An index is created on the **doctor_id** field in the appointments collection, which significantly reduces query execution time during search and aggregation operations. By minimizing full collection scans, indexing enhances the overall efficiency and scalability of the database. It is particularly beneficial in handling large datasets, where frequent queries based on doctor_id are performed. This approach demonstrates how proper indexing strategies can improve system responsiveness and support high-performance data processing in real-world applications.

"Indexing reduces search time by avoiding full data scans and makes queries faster and more efficient."



Importance:
- Improves query speed
- Reduces search time
- Efficient for large datasets

CONCLUSION

This project highlights the efficient utilization of **MongoDB** for managing and analyzing hospital data in a scalable environment. It integrates core database operations such as CRUD, aggregation, and indexing to ensure optimized data handling and performance. The use of embedded documents enhances data retrieval efficiency, while indexing significantly improves query execution speed. Overall, the system demonstrates a robust and flexible approach to handling real-world healthcare data, making it suitable for practical and scalable applications.

THANK YOU

wwww.reallygreatsite.com